



MANUAL DE SISTEMA

**Sistema Web para el Control de Abordajes en las
Comunidades Atendidas por la Fundación Tejiendo
Redes**

Reingeniería del Sistema de Registro

Versión 2.0

Elaborado por:

Equipo de Reingeniería
Universidad Fermín Toro
Facultad de Ingeniería
Escuela de Computación

AGOSTO 2026

CONTROL DE VERSIONES

Versión	Fecha	Autor(es) / Equipo	Descripción del Cambio
1.0	Cohorte anterior de Servicio Comunitario	Equipo de desarrollo anterior	Versión inicial del sistema, construida con Next.js, TypeScript, Drizzle ORM y Tailwind CSS
2.0	Junio 2026	Grupo de Reingeniería - Sistema de Registro (Servicio Comunitario UFT)	Reingeniería integral Nuevo modelo de datos sobre MySQL (19 entidades), autenticación JWT con control de acceso por 6 roles, reconstrucción de los módulos de Consultas, Farmacia y Reportes, e incorporación de Antecedentes, Abordaje-Asistencia, Movimientos de Inventario y Geografía normalizada (Estados-Municipios-Parroquias)
2.1	Julio 2026	Grupo de Reingeniería - Sistema de Registro	Elaboración del presente Manual de Sistema

ÍNDICE GENERAL

	pp.
ÍNDICE GENERAL.....	iii
MODO DE USO.....	iv
INTRODUCCIÓN.....	8
OBJETIVO DEL SISTEMA.....	9
ALCANCE.....	9
ARQUITECTURA GENERAL.....	10
DIAGRAMACIÓN DEL SISTEMA.....	11
Diagrama de Casos de Uso por Rol.....	11
Modelo Entidad-Relación.....	12
Diagrama de Módulos y Arquitectura de Carpetas.....	13
Diseño de Base de Datos.....	16
ENTRADAS DEL SISTEMA.....	17
SALIDAS DEL SISTEMA.....	18
EXPLICACIÓN DE LA LÓGICA POR MÓDULO.....	19
Autenticación y Gestión de Usuarios.....	19
Datos Básicos.....	19
Comunidades.....	19
Abordajes y Solicitudes.....	20
Atención en el Abordaje (Abordaje-Asistencia).....	20
Consultas Médicas.....	20
Farmacia.....	20
Mantenimiento y Auditoría.....	21
SEGURIDAD.....	22
REQUERIMIENTOS DEL SISTEMA.....	23
Hardware.....	23
Software.....	23
Red.....	23
Almacenamiento.....	23
Seguridad.....	24
Recursos Humanos.....	24
MANTENIMIENTO Y RECOMENDACIONES A FUTURO.....	25
GLOSARIO TÉCNICO.....	26

MODO DE USO

Para aprovechar este documento de manera eficiente, tenga en cuenta lo siguiente:

Requisitos previos del lector

Este manual asume conocimientos básicos de desarrollo web y bases de datos relacionales. No es necesario haber participado en el desarrollo del sistema, pero conviene tener cierta familiaridad con:

- Desarrollo basado en componentes (React) y enrutamiento por archivos (Next.js App Router).
- Modelado de bases de datos relacionales: entidades, claves primarias y foráneas, normalización.
- Control de versiones con Git y flujos básicos de despliegue continuo.

Los términos más especializados están definidos en el Glosario Técnico, al final del manual, de modo que no haber trabajado antes con alguna de estas tecnologías no impide su comprensión.

Organización del contenido

El manual se estructura en tres bloques progresivos:

- *Fundamentos del sistema:* Objetivo del Sistema, Alcance y Arquitectura General. Responden qué hace el sistema, qué queda fuera de su cobertura actual, y con qué tecnologías está construida cada capa (frontend, backend, base de datos, autenticación).
- *Diseño técnico:* Diagramación del Sistema (casos de uso por rol, modelo entidad-relación, arquitectura de carpetas y diseño de base de datos), Entradas y Salidas del Sistema, y Explicación de la Lógica por Módulo, con ocho subsecciones que detallan el funcionamiento interno de cada módulo, desde la autenticación hasta la farmacia.
- *Operación y continuidad:* Seguridad, Requerimientos del Sistema (hardware, software, red, almacenamiento, seguridad y recursos

humanos) y Mantenimiento y Recomendaciones a Futuro, con las pautas para dar continuidad al proyecto.

El Glosario Técnico, al final del manual, puede consultarse en cualquier momento como referencia independiente, sin necesidad de leer el resto del documento en orden.

Convenciones utilizadas

- *Negrita*: títulos de sección y términos técnicos clave, generalmente la primera vez que se introducen.
- *Cursiva*: numeración y descripción de las figuras (Figura 1, Figura 2, etc.), referenciadas desde el texto que las precede.
- *Nombres de archivo, rutas, variables de entorno y comandos*: se citan tal como aparecen en el código del proyecto (por ejemplo, DATABASE_HOST o npm run db:refresh).
- *Tablas*: resumen información comparativa o estructurada, como las tecnologías por capa, las entradas y salidas por módulo, o las convenciones de nombrado de archivos.
- *Códigos de entidad*: los prefijos como COM-, ABD- o CON- se muestran junto a un ejemplo numérico (COM-001) para ilustrar el formato real generado por el sistema.
- *Referencias cruzadas*: cuando un término clave no se explica en el mismo párrafo donde aparece, su definición se encuentra en el Glosario Técnico.

Cómo interpretar los diagramas técnicos

Los diagramas de la sección Diagramación del Sistema son ilustrativos y deben leerse junto con su descripción textual, no de forma aislada:

- El Diagrama de Casos de Uso por Rol resume qué puede hacer cada uno de los seis roles definidos en el sistema, pero no sustituye la revisión del control de acceso implementado en el middleware.

- El Modelo Entidad-Relación muestra las relaciones principales entre las 19 entidades, sin listar necesariamente todos los atributos de cada una; el detalle completo de columnas está en el código fuente, dentro de la carpeta db/schema.
- El Diagrama de Módulos y Arquitectura de Carpetas refleja la organización del proyecto al momento de redactar este manual (versión 2.0) y puede variar levemente en futuras iteraciones; ante cualquier discrepancia, el código fuente es la referencia autoritativa.

Guía de lectura según su rol

La siguiente tabla orienta la lectura del manual según el perfil de quien lo consulta. No es necesario leer el documento completo si el interés se limita a un aspecto puntual del sistema:

Perfil	Secciones recomendadas y enfoque de lectura
Administrador del sistema	Arquitectura General, Seguridad, Requerimientos del Sistema, Mantenimiento y Recomendaciones a Futuro. Le interesa sobre todo cómo aprobar usuarios, configurar respaldos y mantener el sistema operativo.
Desarrollador o encargado de mantenimiento	Diagramación del Sistema (completa), Explicación de la Lógica por Módulo, Glosario Técnico. Conviene además revisar ARCHITECTURE.md, DATABASE.md y HANDOVER.md en el repositorio.
Personal de soporte técnico	Entradas y Salidas del Sistema, Seguridad, Requerimientos del Sistema. Útil para diagnosticar incidencias sin necesidad de modificar código.

Documentos relacionados

Este manual es de carácter técnico y se complementa con otros documentos del proyecto:

- **Manual de Usuario:** dirigido al personal operativo de la Fundación; cubre el uso funcional del sistema sin detalles de implementación.
- **ARCHITECTURE.md:** describe la arquitectura y las convenciones de código del proyecto.
- **DATABASE.md:** guía de scripts y del modelo de base de datos.
- **HANDOVER.md:** guía de traspaso para nuevos desarrolladores que se incorporen al mantenimiento del sistema.

Estos tres últimos documentos se encuentran en el repositorio del sistema y se recomienda mantenerlos actualizados en cada nueva iteración, tal como se indica en la sección Mantenimiento y Recomendaciones a Futuro.

INTRODUCCIÓN

El presente Manual de Sistema documenta la arquitectura técnica, el modelo de datos y la lógica operativa del sistema web utilizado por la Fundación Tejiendo Redes para la gestión de sus procesos de atención comunitaria. Este sistema corresponde a la versión reingenierizada por el Grupo de Reingeniería - Sistema de Registro, en el marco del Servicio Comunitario de la Universidad Fermín Toro (UFT), a partir del sistema entregado por una cohorte anterior de la misma asignatura.

A diferencia del Manual de Usuario, dirigido al personal operativo de la Fundación, este documento está dirigido a un público técnico, como los administradores de sistemas, personal de soporte de TI y desarrolladores que den mantenimiento o continuidad al proyecto. Su propósito es que cualquier persona con conocimientos de programación pueda comprender cómo está construido el sistema, cómo se relacionan sus datos y cómo intervenir en él de forma segura, sin necesidad de haber participado en su desarrollo original.

OBJETIVO DEL SISTEMA

El sistema tiene como objetivo centralizar y automatizar el flujo operativo y logístico de la Fundación Tejiendo Redes, permitiendo:

- Registrar y consultar los datos básicos de pacientes, tejedores (voluntarios), médicos, responsables comunitarios, comunidades e instituciones aliadas.
- Planificar y ejecutar abordajes médicos comunitarios, desde la solicitud inicial hasta la asistencia de pacientes en el lugar del operativo.
- Registrar consultas médicas y mantener el historial clínico de cada paciente.
- Controlar el inventario de farmacia y la trazabilidad de la entrega de medicamentos.
- Generar reportes y estadísticas que respalden la toma de decisiones de la Fundación.

ALCANCE

El sistema cubre la totalidad del flujo de datos internos de la Fundación descrito en el objetivo anterior, e incluye:

- Un panel web accesible desde cualquier navegador moderno, con control de acceso diferenciado por rol.
- Una capa de lógica de negocio centralizada (validaciones, generación de códigos, control de duplicados).
- Una base de datos relacional (MySQL) con 19 entidades interrelacionadas.
- Un entorno de pruebas en producción desplegado en Netlify.

Quedan fuera del alcance actual del sistema: la integración directa con los sistemas de organizaciones aliadas (OCHA, UNICEF, ACNUR, entre otras), un módulo de facturación o contabilidad, una aplicación móvil nativa, y el funcionamiento del sistema sin conexión a internet (modo offline).

ARQUITECTURA GENERAL

El sistema sigue una arquitectura cliente-servidor basada en Next.js (App Router), donde el frontend y el backend conviven en un mismo proyecto mediante Server Components y Server Actions. La siguiente tabla resume las tecnologías empleadas en cada capa:

Capa	Tecnología
Frontend / Interfaz	React 19, Next.js 16 (App Router), Tailwind CSS, shadcn/ui, Radix UI, lucide-react (íconos)
Formularios y Validación	React Hook Form + Zod (validación compartida entre cliente y servidor)
Backend / Lógica de negocio	Server Actions de Next.js, separadas en actions/ (escritura) y queries/ (lectura)
Base de Datos	MySQL, gestionada mediante Drizzle ORM (driver mysql2)
Autenticación	JWT (librería jose), bcryptjs para hash de contraseñas, cookies HttpOnly
Estado de interfaz	Zustand (solo modales de edición; los datos de servidor se manejan vía Server Components)
Reportes	jsPDF + jspdf-autotable (PDF), xlsx (CSV/Excel), Recharts (gráficas)
Pruebas	Vitest + React Testing Library + jsdom
Despliegue	Netlify – entorno de pruebas en producción: tejiendoredes-registro.netlify.app

Todas las mutaciones de datos siguen un patrón de validación doble; primero en el cliente (para una respuesta inmediata al usuario) y nuevamente en el servidor, dentro de la Server Action correspondiente, como medida de seguridad ante manipulación del formulario.

DIAGRAMACIÓN DEL SISTEMA

Esta sección presenta de forma gráfica los actores del sistema, el modelo de datos y la organización interna del código, como referencia rápida para cualquier persona que necesite ubicarse dentro del proyecto.

Diagrama de Casos de Uso por Rol

El sistema define seis roles de usuario en la tabla usuarios (admin, superuser, medico, operador, invitado y tejedor), cada uno con visibilidad distinta sobre los módulos del menú principal, controlada de forma centralizada en el archivo de configuración de navegación del proyecto.

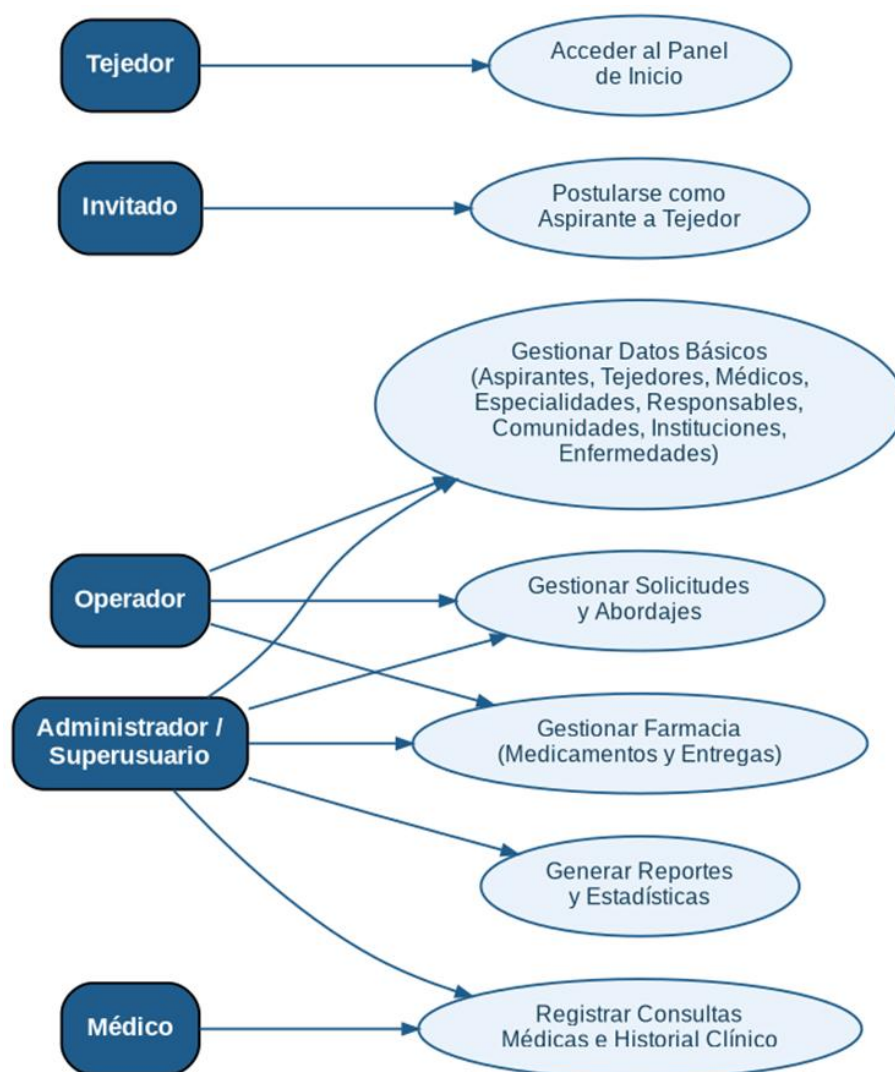


Figura 1. Casos de uso principales por rol de usuario

Es importante señalar que el rol tejedor, aunque existe como valor válido en la base de datos, no tiene actualmente módulos propios asignados en el menú de navegación, por lo que su acceso se limita al panel de inicio. Esto puede deberse a que el rol está reservado para una futura ampliación de funcionalidades específicas para voluntarios, y es un punto que deberá validarse con la Fundación en próximas iteraciones del sistema.

Modelo Entidad-Relación

El modelo de datos se organiza en 19 entidades, agrupables en cuatro núcleos funcionales: (a) el núcleo geográfico y de actores (Geografía, Responsable, Comunidades, Instituciones, Tejedores, Médicos, Especialidades, Usuarios y Aspirantes); (b) el núcleo clínico (Pacientes, Antecedentes, Enfermedades y Consultas); (c) el núcleo logístico de abordajes (Solicitud de Abordaje, Abordaje y Abordaje-Asistencia); y (d) el núcleo de farmacia (Medicamentos, Entregas de Medicamentos y Movimientos de Inventario).

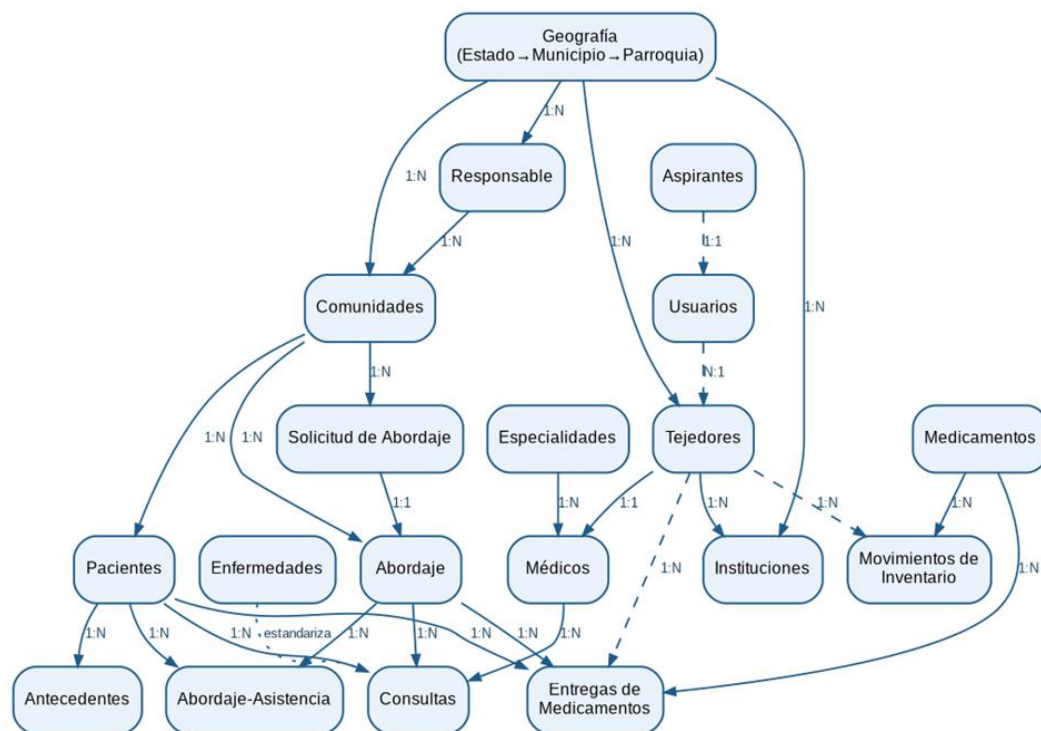


Figura 2. Modelo entidad-relación simplificado del sistema

Cada entidad principal utiliza un código autogenerado con el formato PREFIJO-NNN, calculado de forma segura ante accesos concurrentes mediante una transacción MAX() + FOR UPDATE:

Entidad	Prefijo	Ejemplo
Comunidad	COM-	COM-001
Abordaje	ABD-	ABD-015
Consulta	CON-	CON-042
Medicamento	MED-	MED-003
Especialidad	ESP-	ESP-001

Diagrama de Módulos y Arquitectura de Carpetas

El código del proyecto se organiza por módulos funcionales dentro de src/, separando claramente las rutas (app/), la escritura de datos (actions/), la lectura de datos (queries/), los componentes de interfaz (components/), el acceso a base de datos (db/), las utilidades y la seguridad (lib/) y los esquemas de validación (schemas/).

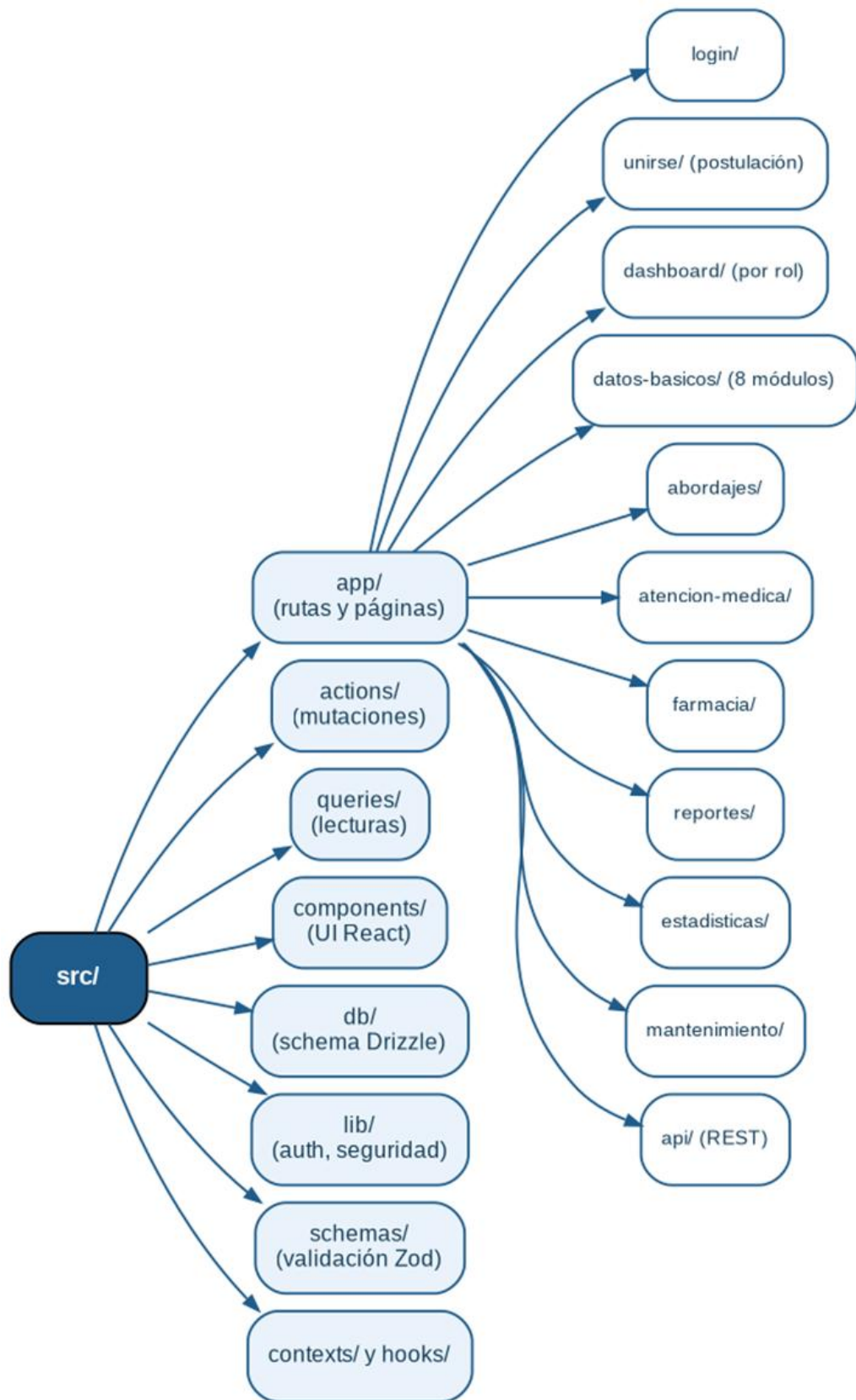
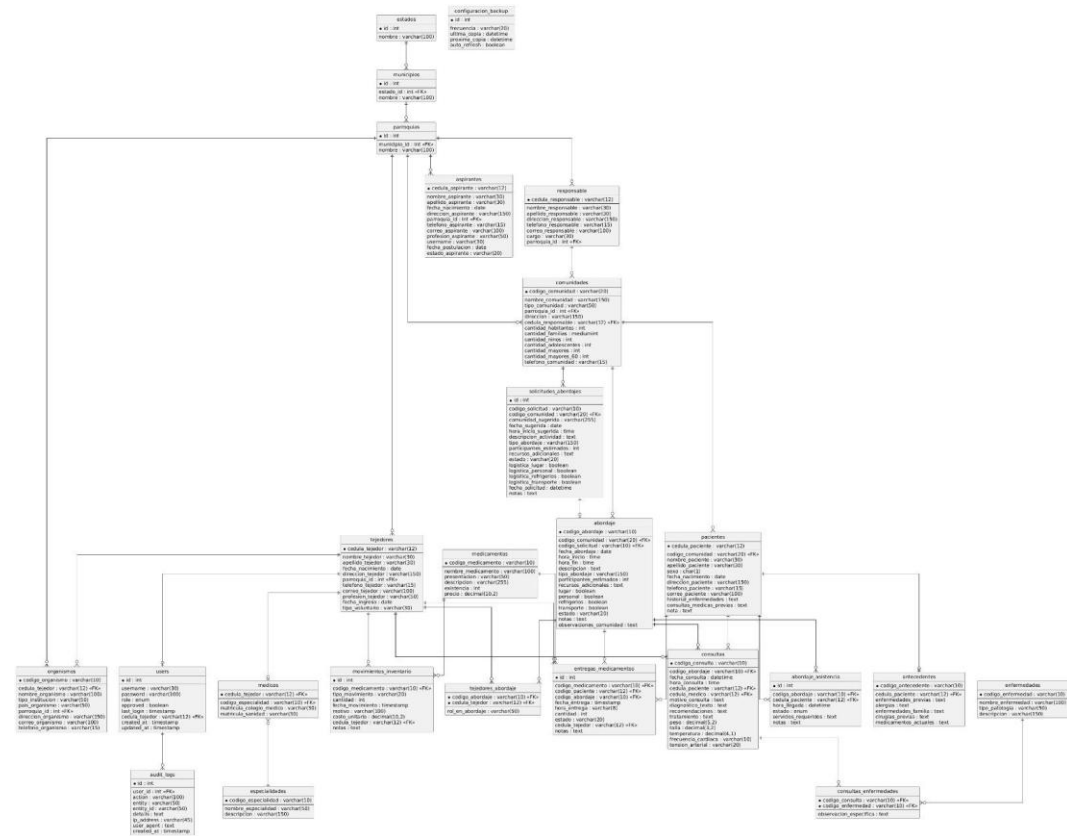


Figura 3. Organización de carpetas y módulos del proyecto

La siguiente tabla resume la convención de nombrado utilizada en el proyecto, documentada en la guía de traspaso (HANDOVER.md) del repositorio:

Tipo de Archivo	Ubicación	Convención de Nombrado
Páginas / Rutas	src/app/[ruta]/page.tsx	Directorio en kebab-case
Server Actions (escritura)	src/actions/[entidad]-actions.ts	kebab-case + sufijo -actions
Server Actions (lectura)	src/queries/[entidad]-actions.ts	kebab-case + sufijo -actions
Componentes	src/components/[categoría]/	PascalCase (ej. PatientForm.tsx)
Schemas de validación	src/schemas/[entidad].ts	kebab-case
Schemas de base de datos	src/db/schema/[entidad].ts	kebab-case

Diseño de Base de Datos



ENTRADAS DEL SISTEMA

Esta sección detalla los datos que el sistema capta para iniciar sus procesos, y qué usuario es responsable de ingresarlos.

Módulo	Dato Capturado	Usuario que lo Ingresa
Aspirantes	Postulación pública de voluntariado (datos personales, profesión)	Público general, vía formulario /unirse
Tejedores	Confirmación de un aspirante aprobado, o registro directo; incluye matrícula y especialidad si la profesión es Médico	Operador / Administrador
Comunidades, Responsables, Instituciones	Datos demográficos y de contacto	Operador / Administrador
Pacientes	Datos básicos, historial de enfermedades y consultas previas	Operador, con verificación de existencia por cédula
Antecedentes	Alergias, cirugías previas, medicamentos actuales, enfermedades familiares	Médico u Operador
Solicitud de Abordaje	Comunidad, fecha sugerida, tipo(s) de abordaje, recursos necesarios	Operador, o comunicación directa con el administrador
Abordaje (confirmación)	Aprobación de la solicitud, fecha y hora definitivas	Administrador / Superusuario
Abordaje-Asistencia	Llegada del paciente y avance de su atención (Triage, Consulta, Farmacia)	Operador, durante el abordaje
Consultas	Motivo, diagnóstico, tratamiento y signos vitales	Médico
Medicamentos	Alta de nuevos medicamentos, existencia y precio	Operador / Administrador
Entregas de Medicamentos	Despacho de un medicamento a un paciente, con responsable de entrega	Operador
Movimientos de Inventario	Entradas, salidas y ajustes de stock (kardex)	Operador / Administrador

SALIDAS DEL SISTEMA

El módulo de Reportes genera cinco salidas principales, cada una con filtros propios y exportación a PDF y CSV:

Reporte	Contenido	Filtros
Abordajes	Código, fecha, descripción, comunidades, pacientes atendidos, hora de inicio y fin	Rango de fechas, estado
Comunidades	Nombre, estado, municipio, habitantes, pacientes tratados, abordajes y consultas realizadas	Búsqueda por nombre, tipo de comunidad
Pacientes	Cédula, nombre, comunidad, fecha de nacimiento, teléfono, correo	Sexo
Morbilidad	Enfermedad, tipo de patología, total de casos, pacientes afectados, porcentaje, última consulta – incluye gráfica de barras	Tipo de patología
Medicamentos	Código, nombre, presentación, existencia, estado de stock	-

Todos los reportes se exportan con encabezado institucional, logotipo de la Fundación, totales calculados al pie de la tabla y nombre de archivo con la fecha de generación.

EXPLICACIÓN DE LA LÓGICA POR MÓDULO

Esta sección complementa los diagramas anteriores con una explicación narrativa de la lógica interna de cada módulo.

Autenticación y Gestión de Usuarios

El acceso al sistema requiere una cuenta aprobada. Un aspirante puede postularse públicamente desde /unirse o mediante el endpoint /api/public/register-tejedor; su solicitud queda en estado pendiente hasta que un administrador la revisa desde el panel de aspirantes o usuarios pendientes y la aprueba. Al aprobar un aspirante, el sistema lo promueve automáticamente a la tabla de tejedores. El inicio de sesión valida las credenciales con bcryptjs y, de ser correctas, genera un token JWT con 24 horas de duración, almacenado en una cookie de sesión HttpOnly.

Datos Básicos

Los formularios de datos básicos aplican dos niveles de validación (cliente y servidor) mediante esquemas Zod compartidos. Antes de insertar un registro, el sistema verifica que la clave primaria (usualmente la cédula) no exista ya en la tabla correspondiente, evitando la duplicidad que afectaba al sistema anterior. Cuando un Tejedor tiene la profesión “Médico”, el sistema exige adicionalmente su matrícula del colegio médico, su matrícula sanitaria y su especialidad, insertando un registro relacionado en la tabla médicos.

Comunidades

Cada comunidad recibe un código mnemotécnico con el formato EST-MUN-PAR-NNN, que codifica el estado, municipio y parroquia a la que pertenece, seguido de un número secuencial. Este esquema reemplaza la codificación ambigua del sistema anterior y permite identificar la ubicación de una comunidad directamente desde su código.

Abordajes y Solicitudes

El flujo de un abordaje comienza con una solicitud, que puede indicar una comunidad ya registrada o el nombre de una comunidad sugerida si esta aún no existe en el sistema. La solicitud permite seleccionar más de un tipo de abordaje simultáneamente y especificar los recursos logísticos necesarios (lugar, personal, refrigerios, transporte) mediante indicadores independientes. Al ser aprobada, la solicitud se vincula de forma 1 a 1 con un registro de abordaje confirmado.

Atención en el Abordaje (Abordaje-Asistencia)

Cuando un paciente llega a un abordaje, se registra su ingreso, el cual mantiene un estado que avanza a medida que el paciente recorre el operativo: En Espera → En Triage → En Consulta → En Farmacia → Finalizado. Un campo adicional permite anotar qué servicios requiere el paciente durante su paso por el abordaje.

Consultas Médicas

Cada consulta queda asociada a un paciente, a un médico y, opcionalmente, a un abordaje específico. El médico registra el motivo de consulta, el diagnóstico narrativo, las recomendaciones, el tratamiento y los signos vitales del paciente (peso, talla, temperatura, frecuencia cardíaca y tensión arterial). El catálogo de enfermedades se utiliza como referencia para estandarizar la terminología usada en los diagnósticos.

Farmacia

El módulo de Farmacia distingue dos flujos: la entrega directa de un medicamento a un paciente, que registra la cantidad, la fecha y al tejedor responsable de la entrega; y el control de inventario a nivel de almacén (kardex), que registra entradas, salidas y ajustes de stock junto con su costo unitario. Esta separación permite auditar tanto lo que se dispensa a cada paciente como los movimientos generales del stock de medicamentos.

Mantenimiento y Auditoría

El sistema incluye una configuración de copias de seguridad, donde se define la frecuencia de respaldo (manual, cada tres días, semanal o mensual) y si este se ejecuta automáticamente. De forma independiente, un registro de auditoría almacena las acciones sensibles del sistema (inicios de sesión, aprobaciones de tejedores, eliminación de usuarios, entre otras) junto con el usuario responsable, la dirección IP y el navegador utilizado, como mecanismo de trazabilidad y seguridad.

SEGURIDAD

La seguridad del sistema se apoya en varios mecanismos complementarios:

- **Autenticación JWT:** librería jose, algoritmo HS256, expiración de 24 horas, almacenado en cookie HttpOnly y Secure.
- **Contraseñas protegidas:** hash bcrypt (bcryptjs); nunca se almacenan en texto plano.
- **Middleware central:** en cada solicitud aplica, en orden: límite de intentos (5 inicios de sesión cada 15 minutos, 3 registros por hora), validación de tokens CSRF en solicitudes POST, validación de la sesión JWT, protección de rutas según el rol (RBAC) y encabezados de seguridad (CSP, HSTS, X-Frame-Options).
- **Sanitización de entradas:** limpieza de las entradas de usuario contra ataques XSS.
- **Consultas preparadas:** Drizzle ORM utiliza prepared statements, que previenen la inyección SQL.
- **Integridad referencial:** claves foráneas configuradas explícitamente en el esquema de base de datos.
- **Auditoría:** registro de acciones sensibles con usuario, IP y navegador, para trazabilidad forense.
- **Aprobación manual:** toda cuenta nueva requiere aprobación de un administrador antes de obtener acceso.

REQUERIMIENTOS DEL SISTEMA

Hardware

Al tratarse de una aplicación desplegada en Netlify (plataforma como servicio), no se requiere un servidor físico dedicado para el frontend ni el backend. Sin embargo, la base de datos MySQL sí requiere un servicio de hosting propio (un proveedor gestionado de bases de datos o un servidor dedicado). Las estaciones de trabajo cliente solo requieren un computador o dispositivo móvil con navegador web moderno y acceso a internet.

Software

Para el entorno de desarrollo se requiere Node.js (versión compatible con Next.js 16, recomendándose Node 20 LTS o superior), un servidor MySQL 8 o superior, y npm como gestor de paquetes. Las variables de entorno de conexión (DATABASE_HOST, DATABASE_PORT, DATABASE_USER, DATABASE_PASSWORD, DATABASE_NAME) deben definirse en un archivo .env.local, el cual no debe subirse al repositorio.

Red

Se requiere una conexión a internet estable, tanto para acceder a la aplicación en producción (Netlify) como para que el backend se comuniquen con la base de datos, especialmente si esta se aloja en un servicio remoto.

Almacenamiento

El crecimiento del almacenamiento depende del volumen de pacientes, abordajes, consultas y movimientos de farmacia registrados. Se recomienda establecer una política de respaldo periódico, configurable desde el módulo de Mantenimiento del sistema, así como respaldos adicionales fuera del proveedor principal.

Seguridad

Se recomienda mantener el archivo de variables de entorno fuera del control de versiones, rotar periódicamente las claves usadas para firmar los tokens JWT, y revisar con regularidad el registro de auditoría del sistema.

Recursos Humanos

Se requiere, como mínimo, un administrador del sistema, encargado de aprobar usuarios y gestionar catálogos; un responsable de mantenimiento técnico o desarrollo, para dar continuidad evolutiva al proyecto; y personal de la Fundación capacitado en el uso operativo del sistema, cubierto en el Manual de Usuario.

MANTENIMIENTO Y RECOMENDACIONES A FUTURO

Para dar continuidad a este proyecto, el repositorio del sistema incluye tres documentos técnicos que se recomienda mantener actualizados en cada nueva iteración: ARCHITECTURE.md (arquitectura y convenciones), DATABASE.md (guía de scripts y modelo de base de datos) y HANDOVER.md (guía de traspaso para nuevos desarrolladores). A partir de ello, se recomienda:

- Utilizar siempre `npm run db:refresh` (migraciones versionadas) en producción, evitando `npm run db:push`, que no deja trazabilidad y puede ejecutar cambios destructivos.
- Seguir la convención de Conventional Commits (`feat:`, `fix:`, `refactor:`, `docs:`) para mantener un historial de cambios legible.
- Al agregar una nueva entidad, seguir los pasos documentados en la guía de traspaso: esquema, registro en el índice, relaciones, migración, esquema de validación Zod y Server Actions correspondientes.
- Revisar periódicamente el rol tejedor, actualmente sin módulos propios asignados en el menú, para decidir si se le asignará una vista propia en una futura versión.
- Evaluar a futuro la incorporación de notificaciones automáticas, una aplicación móvil complementaria, o la ampliación del módulo de Estadísticas.

Este manual, junto con el Manual de Usuario, debe conservarse y entregarse a quien continúe con el mantenimiento del proyecto, como parte del proceso de traspaso del sistema.

GLOSARIO TÉCNICO

Abordaje Comunitario

Operativo desplegado en terreno por la Fundación para brindar atención médica, entrega de medicamentos y servicios comunitarios a una población específica.

App Router

Paradigma de enrutamiento de Next.js basado en la estructura de carpetas `src/app/`, que aprovecha las funcionalidades avanzadas de React 19 como los Server Components.

Conventional Commits

Convención estandarizada para estructurar los mensajes de confirmación de cambios (commits) en Git (`feat:`, `fix:`, `docs:`), facilitando la lectura del historial de desarrollo.

CSRF (Cross-Site Request Forgery)

Tipo de ataque de falsificación de peticiones en sitios cruzados, que el sistema previene mediante tokens de validación en las solicitudes que modifican datos.

Hash / Bcrypt

Algoritmo de encriptado unidireccional utilizado para transformar contraseñas en cadenas irreversibles antes de almacenarlas en la base de datos.

HttpOnly Cookie

Galleta de sesión enviada por el servidor que no puede ser accedida ni manipulada mediante scripts del cliente (JavaScript), protegiendo tokens sensibles como los JWT contra ataques XSS.

Integridad Referencial

Regla de base de datos relacional que garantiza que las relaciones entre tablas se mantengan válidas (por ejemplo, impidiendo registros huérfanos mediante claves foráneas).

JWT (JSON Web Token)

Token firmado digitalmente que permite mantener la sesión de un usuario autenticado de forma segura sin almacenar el estado en el servidor.

Kardex

Registro cronológico de entradas, salidas y ajustes de inventario de un almacén.

Middleware

Capa de software que intercepta cada solicitud HTTP antes de llegar a su destino final, utilizada en este proyecto para autenticación, seguridad (CSP, HSTS) y control de acceso.

Migración

Archivo versionado que describe un cambio estructurado e incremental en la base de datos.

ORM (Object-Relational Mapping)

Capa de software que permite interactuar con la base de datos utilizando código orientado a objetos en lugar de SQL directo; en este proyecto, Drizzle ORM.

PaaS (Platform as a Service)

Plataforma como servicio (como Netlify) que proporciona un entorno donde desplegar la aplicación sin gestionar infraestructura o servidores físicos directamente.

Prepared Statements (Consultas Preparadas)

Técnica de ejecución de consultas SQL que separa la estructura del comando de los datos de entrada, eliminando vulnerabilidades de inyección SQL.

Rate Limiting

Mecanismo que limita el número de solicitudes que un mismo usuario o dirección IP puede realizar en un período de tiempo determinado para prevenir abusos.

RBAC (Role-Based Access Control)

Control de acceso basado en roles; determina qué módulos, vistas y acciones puede ver o ejecutar cada tipo de usuario en el sistema.

Seed

Proceso de inserción automática de datos iniciales o de prueba en la base de datos.

Server Action

Función de Next.js que se ejecuta directamente en el servidor y puede ser invocada desde componentes de React para gestionar mutaciones de datos.

Server Components (RSC)

Componentes de React que se ejecutan e renderizan exclusivamente en el servidor, reduciendo el volumen de código JavaScript enviado al navegador del cliente.

Triage

Proceso de clasificación y evaluación inicial de pacientes para determinar el orden y tipo de atención médica que requieren durante un abordaje.

Zod

Librería de validación de esquemas de datos orientada a TypeScript, utilizada tanto en el cliente como en el servidor para asegurar la integridad de los formularios.

Zustand

Librería liviana de gestión de estado global en el cliente, empleada en el sistema para coordinar la apertura y cierre de ventanas modales de edición.